

Contents

□ Shortest Path Algorithms — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	8
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	9
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	10
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	13
SECTION 16 — Optimization Techniques	14
SECTION 17 — Multiple Language Templates	14
SECTION 18 — Dry Runs	17
SECTION 19 — Advanced Concepts	17
SECTION 20 — Senior Engineer Insights	18
SECTION 21 — Cheat Sheet	18
SECTION 22 — Revision Notes (5-Minute Review)	18
SECTION 23 — Practice Roadmap	18
SECTION 24 — Memory Tricks	19
SECTION 25 — Final Summary	19

□ Shortest Path Algorithms — Complete Interview Handbook

Pattern #21 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Graph section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Shortest Path algorithms find the minimum-cost route between nodes in a **weighted** graph. **Dijkstra's algorithm** greedily expands from the source, always finalizing the currently-closest unvisited node next (using a min-heap), and works only when all edge weights are **non-negative**. **Bellman-Ford** relaxes every edge repeatedly ($V-1$ times), tolerating **negative weights** and detecting negative cycles, at the cost of higher complexity. **Floyd-Warshall** computes **all-pairs** shortest paths via dynamic programming over intermediate nodes.

1.2 Why Was This Pattern Invented?

Plain BFS finds shortest paths only when every edge costs the same (unweighted). Real-world graphs (road networks, network latency, currency exchange) have **varying edge costs** — BFS's level-synchronized guarantee breaks down entirely once edges have different weights. Dijkstra's algorithm generalizes BFS's "always expand the closest frontier next" idea using a priority queue instead of a plain queue, restoring the same greedy correctness guarantee for non-negative weighted graphs. Bellman-Ford and Floyd-Warshall extend this further to handle negative weights and all-pairs queries respectively.

1.3 Real Intuition Behind The Pattern

Imagine **planning a road trip where different roads have different travel times** — you can't just count "number of roads traveled" (that's BFS/unweighted thinking); you need to track actual accumulated travel time and always investigate the currently-closest unexplored city next, since anything closer could still be reached via a cheaper combination of roads you haven't fully explored yet.

1.4 Mental Model

"What's the cheapest way to reach every other node from here, without re-visiting a node with worse information?" Dijkstra's greedily commits to a node's shortest distance once it's popped from the min-heap (a node's true shortest distance is finalized in non-decreasing order, since all edge weights are non-negative). Bellman-Ford instead repeatedly "relaxes" every edge, updating best-known distances, until no further improvement is possible — this is robust even with negative weights because it doesn't commit early.

1.5 Visual Explanation

Graph (weighted, directed): $A \rightarrow B(4)$, $A \rightarrow C(1)$, $C \rightarrow B(2)$, $B \rightarrow D(1)$, $C \rightarrow D(5)$

Dijkstra's from A:

```
dist = {A:0, B:∞, C:∞, D:∞}
```

```
minHeap=[(0,A)]
```

```
pop (0,A): relax A→B: dist[B]=4; relax A→C: dist[C]=1 → heap=[(1,C),(4,B)]
```

```
pop (1,C): relax C→B: dist[B]=min(4,1+2)=3; relax C→D: dist[D]=1+5=6 → heap=[(3,B),(4,B),(6,D)]
```

```
pop (3,B): relax B→D: dist[D]=min(6,3+1)=4 → heap=[(4,B)(stale),(4,D),(6,D)(stale)]
```

```
pop (4,B): already finalized (stale entry), skip
```

```
pop (4,D): finalize D=4
```

Final: $\text{dist} = \{A:0, B:3, C:1, D:4\}$

1.6 Simple Analogy

Dijkstra's algorithm is like **a search-and-rescue team that always investigates the currently-nearest known lead first** — once you've confirmed the actual shortest distance to a location

(popped it from the priority queue), you never need to reconsider it, because anything reachable through unexplored, farther leads can only be equal or farther still (guaranteed by non-negative weights).

1.7 When Should I Immediately Think About Using This Pattern?

- “Shortest path” with **weighted** edges (not unit-cost) and **non-negative** weights → Dijkstra’s.
- “Shortest path” with **possible negative weights**, or need to **detect a negative cycle** → Bellman-Ford.
- “Shortest path between **every pair** of nodes” in a small graph → Floyd-Warshall.
- “Network delay time,” “cheapest flights within K stops,” “path with minimum effort/maximum probability.”

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“shortest path,” weighted edges	Dijkstra’s (non-negative)
“negative weights,” “arbitrage,” “can you detect a negative cycle”	Bellman-Ford
“all pairs shortest path”	Floyd-Warshall
“network delay time”	Dijkstra’s directly
“cheapest flights within K stops”	Modified Bellman-Ford (or BFS with level limit)
“minimum effort path,” “maximum probability path”	Dijkstra’s variant (different relaxation function)

2.2 Hidden Hints

Constraints explicitly mentioning **edge weights** (not just “edges”) is the strongest tell that plain BFS won’t suffice — weighted edges always require Dijkstra’s, Bellman-Ford, or Floyd-Warshall depending on additional constraints (negative weights, all-pairs, stop limits).

2.3 Interview Clues

Interviewer specifies costs/weights/times on edges (not just connections), or explicitly mentions “some edges could be negative” — the latter is a direct signal to rule out Dijkstra’s and use Bellman-Ford instead.

2.4 Common Trick Words

“Minimum cost,” “cheapest,” “fastest,” “with at most K stops” (bounds the number of edges used, requiring a modified relaxation approach), “probability” (multiplicative instead of additive edge weights — requires a max-heap and multiplication instead of a min-heap and addition).

2.5 What Interviewers Expect

Correct algorithm selection based on constraints (non-negative → Dijkstra’s; negative possible → Bellman-Ford; all-pairs and small graph → Floyd-Warshall), correct use of a min-heap with **stale-entry handling** in Dijkstra’s (a node may be pushed multiple times with different distances; only the first pop with the smallest distance is the finalized one), and awareness of complexity trade-offs.

2.6 When NOT To Use This Pattern

- Edge weights are **all equal** (unweighted) — plain BFS (Pattern #18) suffices and is simpler/faster.
 - You need a **minimum spanning tree** (connecting all nodes cheaply), not shortest path from a source — that's Prim's/Kruskal's algorithm (Union-Find, Pattern #19), a related but distinct problem.
 - The graph is **too large** for Floyd-Warshall's $O(V^3)$ all-pairs computation — use Dijkstra's from each source individually ($O(V \times E \log V)$) if only some sources are needed, or accept that all-pairs on huge graphs is often infeasible without approximation.
-

SECTION 3 — Decision Framework

Are edge weights UNIFORM (unweighted)?

Yes → USE PLAIN BFS (Pattern #18) - simpler, $O(V+E)$

No (weighted edges)

Can edge weights be NEGATIVE?

Yes → USE BELLMAN-FORD (handles negative weights, detects negative cycles) - $O(V \times E)$

No (all non-negative)

Do you need shortest path from a SINGLE source to all/some destinations?

Yes → USE DIJKSTRA'S ALGORITHM (min-heap based) - $O((V + E) \log V)$

No

Do you need shortest path between EVERY PAIR of nodes, and the graph is SMALL ($V \sim 400-500$)?

Yes → USE FLOYD-WARSHALL (DP over intermediate nodes) - $O(V^3)$

No → Consider running Dijkstra's from every source individually instead (better for sparse, large)

Why: The choice among these three algorithms is driven entirely by two factors: whether negative weights are possible (rules out Dijkstra's, requiring Bellman-Ford) and whether you need single-source or all-pairs results (driving the Dijkstra's-per-source vs. Floyd-Warshall trade-off, which depends on graph size and density).

SECTION 4 — Why This Pattern Works

Dijkstra's Correctness Proof: *Invariant:* when a node is popped from the min-heap (and thus finalized), its recorded distance is its true shortest distance from the source. *Base case:* the source itself, with distance 0, is trivially correct. *Inductive step:* assume every previously-finalized node has the correct shortest distance. The next node popped, u , has the smallest tentative distance among all unfinalized nodes. Since all edge weights are **non-negative**, any path to u through a yet-unfinalized node w would require $\text{dist}[w] + (\text{non-negative edge}) \geq \text{dist}[w] \geq \text{dist}[u]$ (since u was chosen as the minimum among unfinalized nodes) — so no cheaper path to u can exist through an unfinalized node, confirming u 's tentative distance is indeed its true shortest distance. *Termination:* every reachable node is eventually finalized with its correct shortest distance. **QED.**

(Crucially, this proof **breaks** with negative weights, since a negative edge could make a path through a “farther” unfinalized node cheaper than the currently smallest tentative distance — this is exactly why Dijkstra’s requires non-negative weights.)

Bellman-Ford Correctness Proof: *Claim:* after relaxing all edges $V-1$ times, $\text{dist}[v]$ holds the true shortest distance for every node v (assuming no negative cycle reachable from the source). *Proof sketch:* any shortest path in a graph with V nodes uses at most $V-1$ edges (a simple path can’t revisit a node without forming a cycle, and a cycle can only help if negative — checked separately). Each full pass over all edges guarantees that if the shortest path to some node uses k edges, its distance is correctly computed after k passes (by induction on path length) — so $V-1$ passes suffice for all nodes. A V th pass that still finds an improvement indicates a **negative cycle** reachable from the source (since no valid simple shortest path can improve further).

Floyd-Warshall Correctness Proof: *DP formulation:* $\text{dist}[i][j]$ is updated as $\min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$ for every intermediate node k , processed in an outer loop over all k from 1 to V . *Invariant:* after processing intermediate node k , $\text{dist}[i][j]$ holds the shortest path from i to j using only nodes $\{1, \dots, k\}$ as intermediate stops. *Inductive step:* either the shortest path using nodes up to k doesn’t use k at all (unchanged from the previous iteration) or it does, in which case it decomposes into a shortest $i \rightarrow k$ path and a shortest $k \rightarrow j$ path, both already correctly computed using intermediates up to $k-1$. *Termination:* after processing all V nodes as intermediates, $\text{dist}[i][j]$ is the true all-pairs shortest distance. **QED.**

SECTION 5 — Algorithm Framework

5.1 General Template — Dijkstra’s Algorithm

```
function dijkstra(graph, source, n):
    dist = array of infinity, size n; dist[source] = 0
    minHeap = [(0, source)]

    while minHeap is not empty:
        (d, u) = minHeap.popMin()
        if d > dist[u]: continue                # stale entry, skip

        for (v, weight) in graph.neighbors(u):
            if dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight
                minHeap.push((dist[v], v))

    return dist
```

5.2 General Template — Bellman-Ford

```
function bellmanFord(edges, source, n):
    dist = array of infinity, size n; dist[source] = 0

    for i in range(0, n - 1):
        for (u, v, weight) in edges:
            if dist[u] != infinity and dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight

    # nth pass: check for negative cycle
    for (u, v, weight) in edges:
        if dist[u] != infinity and dist[u] + weight < dist[v]:
            return "NEGATIVE CYCLE DETECTED"
```

```
return dist
```

5.3 General Template — Floyd-Warshall

```
function floydWarshall(dist, n):                                # dist[i][j] pre-initialized with direct edge weights
    for k in range(0, n):
        for i in range(0, n):
            for j in range(0, n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

5.4 Interview Thinking Process

1. "This has weighted edges — plain BFS won't work. Are negative weights possible?"
2. "If non-negative, I'll use Dijkstra's with a min-heap, always finalizing the currently-closest unvisited node next."
3. "If negative weights are possible, I'll use Bellman-Ford, relaxing all edges $V-1$ times, with a final pass to detect negative cycles."
4. "If I need all-pairs shortest paths and the graph is small, I'll use Floyd-Warshall's $O(V^3)$ DP over intermediate nodes."
5. "I'll handle stale heap entries in Dijkstra's by skipping any popped entry whose recorded distance exceeds the current best known distance."

SECTION 6 — Time & Space Complexity

Algorithm	Time	Space	Handles Negative Weights?
Dijkstra's (min-heap)	$O((V + E) \log V)$	$O(V + E)$	No
Bellman-Ford	$O(V \times E)$	$O(V)$	Yes (and detects negative cycles)
Floyd-Warshall	$O(V^3)$	$O(V^2)$	Yes (but not negative cycles reachable in a way that breaks shortest path definition — must be checked separately via diagonal negativity)

Why Dijkstra's needs a heap: without a priority queue, finding the minimum-distance unfinalized node naively costs $O(V)$ per iteration, giving $O(V^2)$ total — the min-heap reduces this to $O(\log V)$ per extraction, at the cost of $O(\log V)$ per edge relaxation (each potentially pushing a new heap entry), giving the combined $O((V+E) \log V)$ bound.

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Disconnected graph	Target unreachable from source	<code>dist[target]</code> remains infinity — correctly represents “unreachable”
Negative edge weight with Dijkstra’s	Any negative edge present	Dijkstra’s produces INCORRECT results silently — must recognize this and switch to Bellman-Ford
Negative cycle reachable from source	Cycle with net-negative total weight	Bellman-Ford’s Vth pass detects continued improvement — must explicitly check and report this
Self-loops	Edge from a node to itself	Correctly ignored if non-negative (never improves distance); if negative, contributes to negative-cycle detection
Single node, no edges	Trivial graph	<code>dist[source] = 0</code> , all others infinity (or undefined if only one node exists)
Multiple edges between the same pair of nodes	Parallel edges with different weights	Relaxation naturally handles this — only the minimum-weight edge ever “wins”
Floyd-Warshall with disconnected pairs	No path between <i>i</i> and <i>j</i>	<code>dist[i][j]</code> remains infinity throughout, correctly representing unreachability

Common mistakes: applying Dijkstra’s to a graph with negative edges without realizing it silently produces wrong answers (not a crash — this is a dangerous, hard-to-detect bug); forgetting the stale-entry check in Dijkstra’s (`if d > dist[u]: continue`), which doesn’t break correctness but wastes time re-processing already-finalized nodes; forgetting to check for negative cycles explicitly in Bellman-Ford (just running $V-1$ passes without the validation Vth pass).

SECTION 8 — Pros & Cons

Advantages: Dijkstra’s is efficient ($O((V+E) \log V)$) for the common non-negative-weight case; Bellman-Ford handles negative weights and detects negative cycles, a capability Dijkstra’s fundamentally lacks; Floyd-Warshall elegantly computes all-pairs shortest paths in a simple triple-nested loop. **Disadvantages:** Dijkstra’s fails silently (not loudly) on negative weights — a dangerous property requiring explicit awareness; Bellman-Ford’s $O(V \times E)$ is significantly slower than Dijkstra’s for large non-negative-weight graphs; Floyd-Warshall’s $O(V^3)$ is infeasible for large graphs (practically limited to $V \leq \sim 400-500$). **Trade-offs:** Dijkstra’s (fast, non-negative only) vs. Bellman-Ford (slower, handles negative weights/cycles) vs. Floyd-Warshall (all-pairs, small graphs only) — the correct choice is entirely determined by weight sign and single-source vs. all-pairs requirements. **Limitations:** None of these three natively handle **dynamic** graphs (edges changing frequently) efficiently — that requires more specialized incremental shortest-path data structures. **Inefficient when:** using Bellman-Ford when Dijkstra’s would suffice (unnecessarily slower); using Floyd-Warshall for single-source queries (unnecessarily computes all pairs when only one is needed).

SECTION 9 — Real World Applications

Domain	Application
Google Maps / Uber / Grab	Shortest/fastest route computation over weighted road networks (travel time as edge weight)
Networking	OSPF and other link-state routing protocols use Dijkstra's algorithm directly to compute shortest routing paths
Amazon	Warehouse-to-customer delivery route optimization with weighted distance/time edges
Financial Systems	Currency arbitrage detection uses Bellman-Ford's negative-cycle detection (converting exchange rates to log-weighted edges)
Telecommunications	Network latency/delay minimization (Network Delay Time is literally a Dijkstra's problem)
Airline Systems	Cheapest flight itinerary computation with layover/stop constraints (modified Bellman-Ford/Dijkstra's)
Game Development	Pathfinding for NPCs/units over weighted terrain (though A* is a common heuristic-guided extension)
Operating Systems	Certain resource-allocation and scheduling algorithms use shortest-path reasoning over dependency-weighted graphs
Robotics	Weighted grid/graph pathfinding for autonomous navigation with variable terrain costs
Supply Chain Optimization	Multi-hop shipping route cost minimization across a weighted logistics network

SECTION 10 — Interview Strategy

How seniors answer: They immediately check for negative weights before choosing an algorithm, explicitly state Dijkstra's non-negative-weight requirement and *why* (the greedy finalization proof breaks with negative edges), and correctly implement stale-entry handling in the min-heap.

How juniors answer: They sometimes apply Dijkstra's without checking for negative weights, producing silently incorrect results, or they use Bellman-Ford by default even when Dijkstra's would be sufficient and faster, missing the opportunity to demonstrate nuanced algorithm selection.

Typical follow-ups: "What if some edge weights are negative — does your solution still work?" "How would you detect a negative cycle?" "How would you extend this to find all-pairs shortest paths?" "What's the complexity difference between Dijkstra's and Bellman-Ford, and when would you choose one over the other?"

Optimization questions: "Can you use a Fibonacci heap to improve Dijkstra's complexity?" (Theoretically to $O(E + V \log V)$, rarely implemented in interviews but worth mentioning as an advanced awareness point.)

SECTION 11 — Pattern Variations

Variation	Description	Example
Dijkstra's (Single-Source, Non-Negative)	Standard shortest path from one source	Network Delay Time
Bellman-Ford (Negative Weights)	Handles negative edges, detects negative cycles	Cheapest Flights Within K Stops (with modification)
Floyd-Warshall (All-Pairs)	DP over intermediate nodes	Find the City With the Smallest Number of Neighbors at a Threshold Distance
Dijkstra's with K-Stop Constraint	Modified relaxation bounding edge count used	Cheapest Flights Within K Stops
Dijkstra's with Multiplicative Weights	Max-heap + multiplication instead of min-heap + addition	Path with Maximum Probability
0-1 BFS (Special Case)	Deque-based BFS for graphs with only 0/1 edge weights	Minimum Cost to Make at Least One Valid Path in a Grid (0-1 BFS variant)
A* Search (Heuristic-Guided)	Dijkstra's + admissible heuristic for faster goal-directed search	Advanced pathfinding (rarely required in typical interviews, but worth mentioning)

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Graph BFS (unweighted)	No edge weights considered, all edges cost 1	Unweighted graphs — simpler and faster than any weighted shortest-path algorithm
Union-Find / MST (Kruskal's/Prim's)	Finds a minimum-cost tree connecting ALL nodes, not shortest path between two specific nodes	Need to connect all nodes cheaply, not find point-to-point shortest distance
Dynamic Programming (general)	Floyd-Warshall IS a DP formulation; other shortest-path-like DP problems exist outside graph contexts	Recognize Floyd-Warshall as a graph-specific DP application
Topological Sort + DP	For DAGs specifically, shortest/longest path can be computed in $O(V+E)$ via topological order — faster than Dijkstra's for this special case	Graph is a DAG (no cycles) — exploit this structure instead of general-purpose shortest-path algorithms

Comparison Table

Aspect	Dijkstra's	Bellman-Ford	Floyd-Warshall
Time	$O((V+E) \log V)$	$O(V \times E)$	$O(V^3)$
Negative weights	No	Yes	Yes
Negative cycle detection	No	Yes	Detectable via diagonal check
Scope	Single-source	Single-source	All-pairs

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	N/A (shortest path algorithms rarely appear at pure Easy difficulty)	—
Medium	Direct Dijkstra's/Bellman-Ford application	Network Delay Time, Path with Minimum Effort
Hard	Modified relaxation with constraints, all-pairs	Cheapest Flights Within K Stops, Find the City With the Smallest Number of Neighbors at a Threshold Distance
Very Hard	Multi-objective or probability-based shortest path, advanced combinations	Path with Maximum Probability, Swim in Rising Water, Minimum Cost to Make at Least One Valid Path in a Grid

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Network Delay Time	Medium	Amazon, Meta, Google, Microsoft	Direct Dijkstra's application	Foundational Dijkstra's mechanics
2	Cheapest Flights Within K Stops	Medium	Amazon, Google, Meta	Modified Bellman-Ford with stop-count limit	Constrained shortest path
3	Path with Minimum Effort	Medium	Google, Amazon	Dijkstra's with max-edge-on-path relaxation	Modified relaxation function
4	Path with Maximum Probability	Medium	Google, Amazon	Dijkstra's variant with multiplicative weights	Multiplicative-weight Dijkstra's

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
5	Find the City With the Smallest Number of Neighbors at a Threshold Distance	Medium	Google, Amazon	Floyd-Warshall all-pairs application	All-pairs shortest path mastery
6	Swim in Rising Water	Hard	Google, Amazon	Dijkstra's-style min-heap grid traversal	Cross-pattern (Dijkstra's + Grid)
7	Minimum Cost to Make at Least One Valid Path in a Grid	Hard	Google, Amazon	0-1 BFS / Dijkstra's on a grid	Advanced 0-1 weighted BFS
8	Number of Ways to Arrive at Destination	Medium	Amazon, Google	Dijkstra's + counting shortest paths	Cross-pattern (Dijkstra's + Counting)
9	Cheapest Flights Within K Stops (Bellman-Ford focus variant)	Medium	Amazon	Reinforce Bellman-Ford's layered relaxation	Layered relaxation technique
10	Negative Weight Cycle Detection (custom/interview variant)	Medium	Google, Amazon	Direct Bellman-Ford negative cycle detection	Negative cycle detection mastery
11	Currency Arbitrage Detection (custom/interview variant)	Hard	Google, Amazon (finance-adjacent)	Bellman-Ford negative cycle via log-transformed weights	Advanced real-world application
12	The Maze II (contrast, weighted grid BFS/Dijkstra's)	Medium	Google	Dijkstra's on grid with rolling-distance weights	Grid-based Dijkstra's application
13	Trapping Rain Water II	Hard	Google, Amazon	Min-heap-based Dijkstra's-style boundary expansion	Cross-pattern (Dijkstra's + Heap)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
14	Reachable Nodes In Subdivided Graph	Hard	Google	Dijkstra's with edge-subdivision reasoning	Advanced Dijkstra's variant
15	Second Minimum Time to Reach Destination	Hard	Google, Amazon	Modified BFS/Dijkstra's for second-shortest path	Advanced constrained shortest path
16	Minimum Obstacle Removal to Reach Corner	Hard	Google	0-1 BFS / Dijkstra's variant	Advanced 0-1 weighted BFS
17	All Paths from Source Lead to Destination (contrast)	Medium	Google	Contrast: DFS-based reachability, not shortest path	Pattern-boundary awareness
18	Design Graph With Shortest Path Calculator	Hard	Google, Amazon (system design-adjacent)	Direct Dijkstra's implementation as a reusable system	Applied system design
19	Course Schedule IV (contrast, reachability)	Medium	Google	Contrast: reachability via topological order, not shortest path	Pattern-boundary awareness
20	Shortest Path in a Grid with Obstacles Elimination	Hard	Google, Amazon	BFS with state augmentation (obstacles remaining) — contrast case	State-augmented BFS
21	Shortest Path Visiting All Nodes	Hard	Google, Amazon	BFS over a bitmask state-space (contrast, unweighted)	Cross-pattern (BFS + Bitmask)
22	Minimum Cost to Reach City With Discounts	Hard	Google (advanced)	Dijkstra's with augmented state (discounts remaining)	State-augmented Dijkstra's
23	Find Critical and Pseudo-Critical Edges in MST (contrast)	Hard	Google	Contrast: Kruskal's/Union-Find MST, not shortest path	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
24	Optimize Water Distribution in a Village (contrast, MST)	Hard	Google, Amazon	Contrast: MST via Union-Find, not shortest path	Pattern-boundary awareness
25	Network Becomes Idle	Medium	Google	Dijkstra's/BFS with time-based reasoning layered on top	Applied Dijkstra's with additional computation
26	Frog Position After T Seconds (contrast, DFS/BFS probability)	Medium	Google	Contrast: DFS with probability tracking, not weighted shortest path	Pattern-boundary awareness
27	Path With Maximum Minimum Value (contrast, binary search + BFS)	Medium	Google	Contrast: binary search + BFS/Union-Find, not pure Dijkstra's	Cross-pattern combination awareness
28	Bus Routes (contrast, unweighted BFS on transformed graph)	Hard	Google	Contrast: unweighted BFS, not shortest path with weights	Pattern-boundary awareness
29	Minimum Number of Days to Disconnect Island (contrast)	Hard	Google	Contrast: Max-Flow/Min-Cut adjacent, not shortest path	Pattern-boundary awareness
30	Design a Ride-Sharing Route Optimizer (custom/interview variant)	Hard	Uber, Grab, Careem (system design-adjacent)	Applied Dijkstra's for real-world route optimization	Applied system design

SECTION 15 — Common Mistakes

1. Applying Dijkstra's to a graph with negative edge weights, producing silently incorrect results (not a crash) because the greedy finalization proof requires non-negative weights. *Fix:* always check for negative weights first and switch to Bellman-Ford if present.

2. Forgetting the stale-entry check in Dijkstra's min-heap implementation (if `d > dist[u]`: `continue`), leading to redundant (though not incorrect) re-processing. *Fix*: always include this guard for correctness clarity and efficiency.
3. Running only $V-1$ passes in Bellman-Ford without the additional validation pass to detect negative cycles. *Fix*: always add the extra pass and explicitly check for continued improvement.
4. Using Floyd-Warshall for single-source queries, wastefully computing all-pairs results when only one source is needed. *Fix*: recognize when Dijkstra's/Bellman-Ford (single-source) is the more efficient, appropriately-scoped choice.
5. Incorrect loop ordering in Floyd-Warshall (the intermediate node k MUST be the outermost loop) — swapping loop order breaks the DP's correctness. *Fix*: always structure the triple loop with k outermost, i and j inside.

Why people fail: each algorithm has one critical, easy-to-overlook correctness precondition (Dijkstra's needs non-negative weights; Bellman-Ford needs the extra validation pass; Floyd-Warshall needs the specific k -outermost loop order) — candidates who memorize the code shape without understanding *why* these preconditions matter often apply the wrong algorithm or get subtle implementation details backwards under pressure.

SECTION 16 — Optimization Techniques

- **Time:** Use Dijkstra's instead of Bellman-Ford whenever weights are confirmed non-negative (significant complexity improvement); use 0-1 BFS (a deque-based technique) instead of Dijkstra's when edge weights are restricted to just 0 and 1, achieving $O(V+E)$ instead of $O((V+E) \log V)$.
- **Space:** For Floyd-Warshall, the distance matrix can often be updated in-place without needing a separate "previous iteration" copy, since the DP recurrence only ever improves values monotonically.
- **Readability:** Clearly separate "relax an edge" logic into a named helper function/concept, used consistently across Dijkstra's and Bellman-Ford implementations.
- **Interview performance:** Proactively state which algorithm you're choosing and why (based on negative-weight possibility and single-source vs. all-pairs scope) before writing any code — this framing alone demonstrates strong graph algorithm fluency.

SECTION 17 — Multiple Language Templates

Java

```
public int[] dijkstra(List<int[]>[] graph, int source, int n) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[source] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    pq.offer(new int[]{0, source});
    while (!pq.isEmpty()) {
        int[] top = pq.poll();
        int d = top[0], u = top[1];
        if (d > dist[u]) continue;
        for (int[] edge : graph[u]) {
            int v = edge[0], w = edge[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[]{dist[v], v});
            }
        }
    }
}
```

```

    }
}
return dist;
}

```

JavaScript

```

function dijkstra(graph, source, n) {
    const dist = new Array(n).fill(Infinity);
    dist[source] = 0;
    // Simple array-based priority queue for illustration (use a real heap in production)
    const pq = [[0, source]];
    while (pq.length) {
        pq.sort((a, b) => a[0] - b[0]);
        const [d, u] = pq.shift();
        if (d > dist[u]) continue;
        for (const [v, w] of graph[u] || []) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push([dist[v], v]);
            }
        }
    }
    return dist;
}

```

PHP

```

function dijkstra(array $graph, int $source, int $n): array {
    $dist = array_fill(0, $n, PHP_INT_MAX);
    $dist[$source] = 0;
    $pq = new SplMinHeap();
    $pq->insert([0, $source]);
    while (!$pq->isEmpty()) {
        [$d, $u] = $pq->extract();
        if ($d > $dist[$u]) continue;
        foreach ($graph[$u] ?? [] as [$v, $w]) {
            if ($dist[$u] + $w < $dist[$v]) {
                $dist[$v] = $dist[$u] + $w;
                $pq->insert([$dist[$v], $v]);
            }
        }
    }
    return $dist;
}

```

Python

```

import heapq
def dijkstra(graph, source, n):
    dist = [float('inf')] * n
    dist[source] = 0
    pq = [(0, source)]

```

```

while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]:
        continue
    for v, w in graph.get(u, []):
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            heapq.heappush(pq, (dist[v], v))
return dist

```

Go

```

func dijkstra(graph map[int][][2]int, source, n int) []int {
    dist := make([]int, n)
    for i := range dist {
        dist[i] = math.MaxInt32
    }
    dist[source] = 0
    pq := &PriorityQueue{{0, source}}
    heap.Init(pq)
    for pq.Len() > 0 {
        top := heap.Pop(pq).(Item)
        d, u := top.dist, top.node
        if d > dist[u] {
            continue
        }
        for _, edge := range graph[u] {
            v, w := edge[0], edge[1]
            if dist[u]+w < dist[v] {
                dist[v] = dist[u] + w
                heap.Push(pq, Item{dist[v], v})
            }
        }
    }
    return dist
}
// (Item struct and PriorityQueue heap.Interface implementation assumed defined elsewhere)

```

C++

```

vector<int> dijkstra(vector<vector<pair<int,int>>>& graph, int source, int n) {
    vector<int> dist(n, INT_MAX);
    dist[source] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    pq.push({0, source});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto& [v, w] : graph[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

```

```

    }
    return dist;
}

```

SECTION 18 — Dry Runs

Small Input

See §1.5 for the full Dijkstra’s dry run on $A \rightarrow B(4)$, $A \rightarrow C(1)$, $C \rightarrow B(2)$, $B \rightarrow D(1)$, $C \rightarrow D(5)$, yielding $\text{dist} = \{A:0, B:3, C:1, D:4\}$.

Bellman-Ford Dry Run

Same graph, treated as edge list $[(A,B,4), (A,C,1), (C,B,2), (B,D,1), (C,D,5)]$, source A, $n=4$:

Pass 1: relax $A \rightarrow B$: $\text{dist}[B]=4$; relax $A \rightarrow C$: $\text{dist}[C]=1$; relax $C \rightarrow B$: $\text{dist}[B]=\min(4,1+2)=3$; relax $B \rightarrow D$: $\text{dist}[D]=m$

Pass 2: relax $A \rightarrow B$: no change; relax $A \rightarrow C$: no change; relax $C \rightarrow B$: no change (already 3); relax $B \rightarrow D$: no change

Pass 3: ($n-1=3$ passes total) no further changes

Validation pass: no further improvement found $\rightarrow$ no negative cycle

Final: $\text{dist} = \{A:0, B:3, C:1, D:4\}$ — matches Dijkstra’s result exactly, as expected for non-negative weights

Large Input (Conceptual)

For a graph with 10^5 vertices and 5×10^5 edges, Dijkstra’s with a binary heap costs $O((10^5 + 5 \times 10^5) \times \log(10^5)) \approx 6 \times 10^5 \times 17 \approx 10^7$ operations — feasible within typical time limits; Bellman-Ford on the same graph would cost $O(10^5 \times 5 \times 10^5) = 5 \times 10^{10}$ — far too slow, illustrating why algorithm choice matters enormously at scale.

Corner Case

Disconnected node: if node E has no incoming edges from the source’s reachable component, $\text{dist}[E]$ remains infinity throughout — correctly representing unreachability in both Dijkstra’s and Bellman-Ford.

SECTION 19 — Advanced Concepts

- **0-1 BFS:** when edge weights are restricted to exactly 0 or 1, a deque can simulate Dijkstra’s behavior in $O(V+E)$: push 0-weight edge relaxations to the **front** of the deque, and 1-weight edge relaxations to the **back** — this maintains the same non-decreasing distance-processing order as a full Dijkstra’s, without needing a heap at all.
- **A* Search:** an extension of Dijkstra’s that uses an admissible heuristic function (never over-estimating the true remaining distance) to prioritize exploration toward the goal, often dramatically reducing the number of nodes explored in practice — widely used in pathfinding and games, worth mentioning as an advanced technique even if rarely required to implement from scratch in interviews.
- **Johnson’s Algorithm:** combines Bellman-Ford (to reweight edges, eliminating negative weights via a potential function) with Dijkstra’s (run from every source on the reweighted graph) to solve all-pairs shortest paths in $O(V^2 \log V + VE)$ — better than Floyd-Warshall’s $O(V^3)$ for sparse graphs, an advanced Staff-level topic.
- **State-augmented Dijkstra’s:** for problems like “Cheapest Flights Within K Stops” or “Minimum Cost to Reach City With Discounts,” the graph’s “state” isn’t just the node — it’s $(\text{node}, \text{stopsUsed})$ or $(\text{node}, \text{discountsRemaining})$ — applying Dijkstra’s or BFS over this augmented state space is a powerful generalization technique for constrained shortest-path problems.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize that shortest-path algorithm selection is fundamentally about **matching the algorithm's core assumption (non-negative weights, single-source vs all-pairs, graph size) to the actual problem constraints** — a mismatch (like using Dijkstra's on negative weights) produces silently wrong results, one of the more dangerous classes of bugs because it doesn't manifest as an obvious crash. They're also fluent in generalizing these algorithms via **state augmentation** (treating "node + additional constraint state" as the effective graph node) to solve constrained variants without inventing new algorithms from scratch — a technique that generalizes far beyond shortest-path problems into general state-space search. Interviewers evaluate whether a candidate proactively checks the negative-weight assumption and can correctly reason about why Dijkstra's greedy proof specifically requires it.

SECTION 21 — Cheat Sheet

PATTERN: Shortest Path Algorithms

RECOGNIZE: "shortest path," weighted edges, "cheapest," "minimum cost," "network delay," "negative cycle"

DECISION: unweighted → BFS | non-negative weights, single-source → Dijkstra's | negative weights possible

TEMPLATE (Dijkstra's):

```
dist[source]=0, others=infinity; minHeap=[(0,source)]
while heap: (d,u)=popMin(); if d>dist[u]: continue
    for (v,w) in adj[u]: if dist[u]+w<dist[v]: dist[v]=dist[u]+w; push((dist[v],v))
```

COMPLEXITY: Dijkstra's $O((V+E)\log V)$ | Bellman-Ford $O(V \times E)$ | Floyd-Warshall $O(V^3)$

KEY PROOF: Dijkstra's greedy finalization requires non-negative weights; Bellman-Ford's V-1 passes suffice

WATCH FOR: negative weights silently breaking Dijkstra's, missing the Bellman-Ford validation pass, Floyd-

DOESN'T APPLY WHEN: unweighted (use BFS), need MST not shortest path (use Kruskal's/Prim's), graph is a D

SECTION 22 — Revision Notes (5-Minute Review)

- Unweighted → BFS. Non-negative weighted, single-source → Dijkstra's (min-heap). Negative weights possible → Bellman-Ford (V-1 relax passes + validation). All-pairs, small graph → Floyd-Warshall (k-outermost DP).
 - Dijkstra's greedy proof requires non-negative weights — breaks silently otherwise.
 - Bellman-Ford's extra (Vth) pass detects negative cycles.
 - Floyd-Warshall: k must be the OUTERMOST loop for correctness.
 - 0-1 BFS (deque-based) handles the special case of only 0/1 edge weights in $O(V+E)$.
 - State-augmented Dijkstra's/BFS handles constrained variants (K stops, discounts remaining).
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic Dijkstra's application	Network Delay Time (743)

Stage	Focus	Recommended LeetCode Problems
Intermediate	Modified relaxation, all-pairs	Path with Minimum Effort (1631), Find the City With the Smallest Number of Neighbors at a Threshold Distance (1334)
Advanced	Constrained shortest path, multiplicative weights	Cheapest Flights Within K Stops (787), Path with Maximum Probability (1514)
Expert	State-augmented, 0-1 BFS, advanced hybrid	Swim in Rising Water (778), Minimum Cost to Make at Least One Valid Path in a Grid (1368), Minimum Obstacle Removal to Reach Corner (2290)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Dijkstra: Done when popped (non-negative only).” “Bellman-Ford: Brute-force relax, handles bad (negative) weights.”
 - **Visualization:** A search-and-rescue team always chasing the nearest confirmed lead (Dijkstra’s) versus **patiently re-checking every route V-1 times to be absolutely sure, even if some roads have refunds (negative weights)** (Bellman-Ford).
 - **Recognition shortcut:** Weighted + non-negative + single-source → Dijkstra’s. Negative weights mentioned → Bellman-Ford. All-pairs + small graph → Floyd-Warshall.
-

SECTION 25 — Final Summary

Dijkstra’s algorithm greedily finalizes the closest unvisited node using a min-heap, correct only when all edge weights are non-negative; Bellman-Ford relaxes every edge V-1 times to tolerate negative weights and detect negative cycles; Floyd-Warshall computes all-pairs shortest paths via a DP over intermediate nodes, with the intermediate node strictly required as the outermost loop. The single most important thing to remember forever: **always check for negative edge weights before choosing Dijkstra’s — its greedy correctness proof fundamentally depends on non-negative weights, and using it anyway produces silently wrong answers, not a crash — and remember that Bellman-Ford’s characteristic extra validation pass is what detects negative cycles, not just the V-1 relaxation passes alone.**